



UNICONVERGE TECHNOLOGIES PVT. LTD.

Personal Knowledge OS

RAG-Powered Second Brain

Complete Technical Documentation

Author Abhishek Kumar Shukla	Version 2.0 — May 2026
Role Senior AI Developer	Status Production

Chapter 1: Introduction & Overview

1.1 What is Personal Knowledge OS?

Personal Knowledge OS is a production-grade, AI-powered second brain system that transforms how you interact with your personal documents, notes, code, and knowledge. Instead of manually searching through files or relying on memory, you simply ask questions in plain English via Telegram and receive intelligent, sourced answers drawn from your own documents.

The system uses a technique called Retrieval-Augmented Generation (RAG) — a cutting-edge AI approach that combines the power of vector search with large language models to provide answers that are grounded in your actual documents rather than hallucinated from general training data.

1.2 The Problem It Solves

Knowledge workers accumulate thousands of documents, notes, PDFs, code files, and emails over the course of their work. These resources are rarely revisited — research shows that over 80% of personal knowledge assets are never retrieved after initial creation. The reasons are familiar:

- Files are scattered across different formats and folders
- Search tools can only match exact keywords, not meaning
- There is no way to ask questions across multiple documents simultaneously
- Important insights from old documents are forgotten and rediscovered later
- Accessing files remotely requires knowing exactly where they are

Personal Knowledge OS solves all of these problems with a conversational AI interface that understands the meaning of your questions and finds relevant information across your entire knowledge base.

1.3 Key Features

- Universal file ingestion: PDF, Word, Excel, PowerPoint, code, images, markdown, CSV
- Project-aware retrieval: organises files into named project folders for structured search
- Conversational query: ask questions in plain English via Telegram
- File delivery over Telegram: request any file and receive it directly in the chat
- Multi-source ingestion: Notion pages, Obsidian vault, Gmail threads
- Weekly digest: automatic surfacing of forgotten ideas from your knowledge base
- Three embedding backends: Voyage AI, OpenAI, or Ollama — choose what fits your needs

1.4 Who Is This For?

User Type	Use Case	Example
Researchers	Query papers and notes	What did I find about attention mechanisms in my 2025 papers?
Developers	Search code and specs	How is the OSNet model initialised in my Reid project?
Project Managers	Find proposals and reports	Send me the Q1 budget proposal
Students	Study from own notes	Summarise my notes on transformer architecture
Analysts	Query data documents	What were the key findings in the Excel report?

1.5 Technology Stack

The system is built entirely on open and well-supported technologies:

Component	Technology & Purpose
Language	Python 3.12 — ingestion pipeline, RAG query, file sender
Gateway	OpenClaw (Node.js) — receives Telegram messages, dispatches skills
Vector DB	ChromaDB — stores document embeddings and metadata locally
Embeddings	Ollama (nomic-embed-text) / Voyage AI / OpenAI — converts text to vectors
LLM	Anthropic Claude (claude-haiku-4-5) — synthesises answers from retrieved context
Tunnel	ngrok — exposes Lightning AI to the public internet for Telegram webhooks
Platform	Lightning AI Studio — cloud development and execution environment
Interface	Telegram Bot API — user-facing chat interface

Chapter 2: Core Concepts Explained

2.1 What is RAG? (Retrieval-Augmented Generation)

RAG is the foundational AI technique powering this system. To understand it, consider how a student answers an open-book exam: they don't memorise everything — they read the question, flip to the relevant chapter, read the relevant section, and then write their answer based on what they found.

RAG works the same way. Instead of asking an AI to answer from memorised training data (which may be outdated or hallucinated), RAG first retrieves relevant passages from your own documents, then asks the AI to synthesise an answer based only on those passages.

The three steps of RAG are:

1. Retrieval: find the most relevant document chunks for the user's question
2. Augmentation: add those chunks as context to the prompt
3. Generation: the LLM generates an answer grounded in the retrieved context

Why RAG matters

Without RAG, an AI answers from its training data — which was cut off at a fixed date and does not contain your personal documents. With RAG, the AI answers from YOUR documents in real time, with source attribution so you can verify every claim.

2.2 What is a Vector Embedding?

A vector embedding is a mathematical representation of text as a list of numbers (called a vector). The remarkable property of embeddings is that semantically similar texts produce numerically similar vectors — meaning you can measure how related two pieces of text are by comparing their vectors.

For example, the sentences 'The cat sat on the mat' and 'A feline was resting on a rug' would produce very similar embeddings, even though they share no words. This is called semantic similarity.

In this system, every document chunk is converted to an embedding vector and stored in ChromaDB. When you ask a question, your question is also converted to an embedding, and ChromaDB finds the document chunks whose embeddings are closest to your question's embedding — these are the most semantically relevant chunks.

2.3 What is ChromaDB?

ChromaDB is a vector database — a specialised database designed to store and search embedding vectors at high speed. Unlike traditional databases that search by exact text match or SQL queries, ChromaDB searches by mathematical similarity between vectors.

ChromaDB is used in this project because it runs entirely locally on disk — no external service, no cloud dependency, no API costs. Your data never leaves your machine. It stores not only the vector embeddings but also the original text and metadata (filename, project name, file path, etc.) alongside each vector.

ChromaDB organises data into collections. This project uses a single collection called `knowledge_os`. Within the collection, each record (called a chunk) contains:

- `id`: a unique MD5 hash identifying the chunk
- `embedding`: the vector representation (768 dimensions for Ollama, 512 for Voyage AI)
- `document`: the original text of the chunk
- `metadata`: `filename`, `project_name`, `file_path`, `file_type`, `chunk_index`, `file_size_kb`

2.4 What is Chunking?

A document cannot be embedded as a single unit — embedding models have a maximum input length (typically 8,192 tokens), and more importantly, smaller, focused chunks produce better search results than entire documents.

Chunking is the process of splitting a document into smaller overlapping pieces. This project uses word-based chunking with the following parameters:

Parameter	Value & Explanation
CHUNK_SIZE	400 words — each chunk is approximately 400 words long
CHUNK_OVERLAP	50 words — consecutive chunks share 50 words to preserve context across boundaries
Minimum length	30 characters — chunks shorter than this are discarded as noise

The overlap is critical: without it, a sentence split across two chunks might never be retrieved because neither half alone is meaningful enough to match a query.

2.5 What is OpenClaw?

OpenClaw is a self-hosted AI agent gateway. It acts as the middleman between Telegram (where you type messages) and the Python RAG pipeline (where your knowledge is stored). When you send a message to your Telegram bot, OpenClaw receives it via a webhook, decides which skill to use based on the message content, and executes the corresponding shell command.

OpenClaw is configured with skills — markdown files that describe what triggers the skill and what command to run. This project defines three custom skills: rag-search, file-send, and universal-ingest.

2.6 What is ngrok?

Telegram's servers need a publicly accessible HTTPS URL to send messages to your bot (this is called a webhook). Lightning AI Studio runs on a private cloud server without a public URL by default.

ngrok solves this by creating an encrypted tunnel from a public HTTPS URL (e.g., <https://xxxx.ngrok-free.app>) to your private Lightning AI Studio on port 18789. All webhook requests from Telegram arrive at the ngrok URL and are forwarded to OpenClaw running locally.

When the ngrok tunnel changes (as happens on each new session), the Telegram webhook must be updated to point to the new URL. The startup script handles this automatically.

Chapter 3: System Architecture

3.1 High-Level Architecture

The system consists of two main pipelines: the Ingestion Pipeline (offline, run once per batch of new documents) and the Query Pipeline (online, runs on every Telegram message).

Ingestion Pipeline

This pipeline runs when you add new documents to your knowledge base:

- I. Files placed in my_knowledge/ project folders
- II. universal_ingest.py detects file type and extracts text content
- III. Text is chunked into 400-word overlapping segments
- IV. Each chunk is embedded via Ollama/Voyage/OpenAI into a float vector
- V. Vector + text + metadata stored in ChromaDB on disk

Query Pipeline

This pipeline runs on every Telegram message:

- I. User sends message to Telegram bot
- II. Telegram forwards message to ngrok public URL via webhook
- III. ngrok forwards to OpenClaw Gateway on port 18789
- IV. OpenClaw agent reads message, selects appropriate skill
- V. Skill executes: python src/rag/rag_query.py 'question'
- VI. rag_query.py embeds the question into a vector
- VII. ChromaDB finds top-5 most similar document chunks
- VIII. Chunks + question sent to Claude API for answer synthesis
- IX. Answer returned to OpenClaw, then to Telegram, then to user

3.2 Data Flow Diagram

The complete data flow from user message to response:

INGESTION (Offline)	QUERY (Online)
1. my_knowledge/ files 2. universal_ingest.py 3. Text extraction 4. Chunking (400w / 50w overlap) 5. Embedding (Ollama/Voyage/OpenAI) 6. ChromaDB storage	1. Telegram message 2. Telegram → ngrok → OpenClaw 3. OpenClaw skill selection 4. rag_query.py executes 5. Question embedded 6. ChromaDB similarity search (top-5) 7. Claude synthesises answer 8. Response → Telegram

Chapter 4: Project Structure & File Reference

4.1 Directory Layout

Every file in this project has a specific purpose. The structure is designed to be modular — you can add new ingestion sources, embedding providers, or skills without touching existing code.

personal-knowledge-rag-os/	
├── README.md	Project overview and quick-start guide
├── .env.example	Template for all configuration variables
├── .env	Your actual credentials (never commit this)
├── .gitignore	Excludes credentials, chromadb, and logs
├── requirements.txt	Python package dependencies
├── start.sh	Original startup script (basic)
├── ~/start_bot.sh	Full startup script with all dependencies
├── config/	
│ └── agents/	
│ └── AGENTS.md	OpenClaw agent persona and instructions
├── my_knowledge/	Your documents (gitignored)
│ ├── Person-Reid-Model/	Example project folder
│ │ ├── documents/	Word docs, PDFs for this project
│ │ └── modules/	Python source files
│ └── README.md	Instructions for this folder
├── src/	
│ ├── ingest/	
│ │ ├── universal_ingest.py	Multi-format ingestion pipeline (primary)
│ │ ├── pdf_ingest.py	PDF-only ingestion (legacy)
│ │ ├── notion_ingest.py	Notion pages ingestion
│ │ ├── obsidian_ingest.py	Obsidian markdown vault ingestion
│ │ └── gmail_ingest.py	Gmail threads ingestion
│ ├── embed/	
│ │ └── embedder.py	Embedding provider (Ollama/Voyage/OpenAI)
│ ├── rag/	
│ │ ├── chromadb_store.py	ChromaDB wrapper (store/search/stats)
│ │ └── rag_query.py	Main query engine + file routing
│ ├── telegram/	
│ │ ├── __init__.py	Makes telegram/ a Python package
│ │ └── file_sender.py	File search and Telegram delivery
│ └── skills/	
│ ├── rag-search/SKILL.md	OpenClaw knowledge query skill
│ ├── file-send/SKILL.md	OpenClaw file delivery skill
│ ├── universal-ingest/SKILL.md	OpenClaw ingestion trigger skill
│ └── weekly-digest/SKILL.md	OpenClaw weekly digest skill
├── chromadb/	Vector store data (gitignored)
└── logs/	Runtime logs (gitignored)

Chapter 5: File-by-File Deep Dive

5.1 universal_ingest.py

This is the primary ingestion engine for the entire system. It replaces the individual PDF, Word, and other format-specific ingestors with a single unified pipeline that automatically detects file types and routes them to the correct extractor.

Location

```
src/ingest/universal_ingest.py
```

Purpose

When you drop any file into a project folder under my_knowledge/, running this script will read the file, extract its text content, chunk it, embed it, and store it in ChromaDB with full metadata.

Supported File Types

Extension(s)	Type	Extraction Library
.pdf	PDF document	PyMuPDF (fitz)
.docx, .doc	Word document	python-docx
.xlsx, .xls, .csv	Excel / Spreadsheet	pandas + openpyxl
.pptx, .ppt	PowerPoint	python-pptx
.jpg, .jpeg, .png, .webp, .bmp	Image	Claude Vision API
.py, .js, .ts, .java, .go, .rs, .sh, .yaml, .json, .sql	Code	Plain text read
.txt, .md, .rst	Text / Markdown	Plain text read

Project Folder Detection

The script automatically detects which project a file belongs to based on its folder position:

```
my_knowledge/Person-Reid-Model/modules/config.py → project_name = 'Person-Reid-Model'
my_knowledge/Finance/Q1-report.xlsx → project_name = 'Finance'
my_knowledge/README.md → project_name = 'root'
```

Metadata Stored per Chunk

Metadata Field	Description
source	Always 'universal' — identifies ingestion pipeline
file_type	pdf / word / excel / powerpoint / image / code / text
project_name	Parent folder name (e.g., 'Person-Reid-Model')
filename	Original filename (e.g., 'Project Proposal.docx')
file_path	Absolute path — used by file_sender.py to deliver the file
rel_path	Relative path from my_knowledge/ — used for display

Metadata Field	Description
extension	File extension (e.g., '.docx')
chunk_index	Position of chunk within the file (0, 1, 2, ...)
file_size_kb	File size in kilobytes

Key Configuration Constants

```

KNOWLEDGE_DIR = Path('my_knowledge')    # Root folder to scan
CHUNK_SIZE     = 400                     # Words per chunk
CHUNK_OVERLAP  = 50                     # Overlap words between chunks
MAX_FILE_SIZE  = 25 * 1024 * 1024       # 25 MB maximum file size

```

CLI Usage

```

# Ingest all files from all project folders
python src/ingest/universal_ingest.py

```

```

# Ingest one specific project
python src/ingest/universal_ingest.py --project "Person-Reid-Model"

```

```

# Ingest a single file
python src/ingest/universal_ingest.py --path my_knowledge/report.pdf

```

```

# Delete existing chunks and re-ingest (use when updating files)
python src/ingest/universal_ingest.py --clean --project "Person-Reid-Model"

```

How Image Ingestion Works

Images are handled differently from text files. Since images contain visual information that cannot be extracted as text, the system sends each image to the Claude Haiku Vision API with a detailed extraction prompt. Claude describes the image content, extracts any visible text, and identifies charts, diagrams, or data. This description is then chunked and stored like any other document.

Skip Directories

The ingestor automatically skips `.git/`, `__pycache__`, `node_modules/`, `output/`, `weight/`, `chromadb/`, and `logs/`. This prevents ingesting internal tooling files, trained model weights, or generated outputs.

5.2 embedder.py

The embedding pipeline is the most critical infrastructure component. Every piece of text — both during ingestion and during query — must pass through the embedder to be converted into a vector before ChromaDB can store or search it.

Location

```
src/embed/embedder.py
```

Three Embedding Providers

The embedder supports three providers with automatic priority detection based on which API keys are present in `.env`:

Provider 1: Voyage AI (Recommended)

Voyage AI is a cloud embedding service optimised for use with Claude. It offers 200 million free tokens — more than enough for hundreds of thousands of document chunks. It does not require any local installation and works perfectly on Lightning AI without session restart issues.

```
Model: voyage-3-lite
Dimension: 512 floats per vector
Activation: Set VOYAGE_API_KEY in .env
```

Provider 2: OpenAI

OpenAI embeddings are paid but offer excellent quality. Use this if you already have an OpenAI API key and want the best possible retrieval accuracy.

```
Model: text-embedding-3-small
Dimension: 1536 floats per vector
Activation: Set OPENAI_API_KEY in .env
```

Provider 3: Ollama (Local, Free)

Ollama runs locally on your machine (or Lightning AI Studio) and uses no external API. The nomic-embed-text model must be downloaded once (274 MB). The main drawback on Lightning AI is that Ollama must be reinstalled each session.

```
Model: nomic-embed-text
Dimension: 768 floats per vector
Activation: Set OLLAMA_BASE_URL=http://localhost:11434 in .env
```

Critical Warning: Dimension Mismatch

You cannot mix embeddings from different providers in the same ChromaDB collection. If you switch from Ollama (768 dimensions) to Voyage AI (512 dimensions), you must delete the collection and re-ingest everything. Failing to do so will cause a dimension mismatch error.

Auto-Detection Logic

The `get_embedder()` function reads environment variables and returns the first available provider:

```
if VOYAGE_API_KEY is set and valid → VoyageEmbedder
elif OPENAI_API_KEY is set and valid → OpenAIEmbedder
elif OLLAMA_BASE_URL is set → OllamaEmbedder
else → raises ValueError with setup instructions
```

Retry Logic for Ollama

The Ollama embedder includes a fallback retry mechanism for long texts. It first attempts to embed up to 4,000 characters. If Ollama returns a 500 error (text too long), it retries with 2,000 characters, then 500 characters. This handles large code files and verbose documents gracefully.

5.3 chromadb_store.py

This file is the interface between the rest of the application and ChromaDB. It abstracts the database operations into simple functions so that neither the ingestors nor the query engine need to know the details of ChromaDB's API.

Location

```
src/rag/chromadb_store.py
```

Key Functions

Function	Description
<code>get_collection()</code>	Returns the ChromaDB collection, creating it if it does not exist
<code>search(embedding, n_results, source_filter)</code>	Finds the top-N most similar chunks
<code>get_stats()</code>	Returns total chunk count broken down by source
<code>delete_by_source(source)</code>	Removes all chunks from a specific source

Collection Configuration

ChromaDB uses cosine similarity for vector comparison (configured via the distance function parameter). Cosine similarity measures the angle between two vectors — perfect for text embeddings where the length of the vector should not affect similarity.

```
client = chromadb.PersistentClient(CHROMADB_PATH)
collection = client.get_or_create_collection(
    name=CHROMADB_COLLECTION,
    metadata={'hnsw:space': 'cosine'}
)
```

Source Filtering

When querying, you can optionally filter results to only chunks from a specific source (pdf, notion, obsidian, gmail, universal). This is passed as a where clause to ChromaDB:

```
collection.query(
    query_embeddings=[embedding],
    n_results=5,
    where={'source': 'universal'},
    include=['documents', 'metadatas', 'distances']
)
```

5.4 rag_query.py

This is the brain of the system. Every query from Telegram flows through this file. It handles three distinct modes of operation: standard RAG query, file send routing, and file list routing.

Location

```
src/rag/rag_query.py
```

Intent Detection

Before performing a RAG search, `rag_query.py` analyses the user's question to determine what they actually want:

File Send Intent

Triggered when the message contains phrases like: 'send me', 'share', 'get me', 'fetch', 'forward me', 'attach', 'send the', 'send document'. When detected and a `chat_id` is provided, the request is routed to `file_sender.py` instead of the RAG pipeline.

File List Intent

Triggered when the message contains: 'list files', 'what files', 'show files', 'what documents', 'files in'. Routes to `list_files_in_project()` in `file_sender.py`.

Project Extraction

The query engine attempts to extract a project name from the question using regex patterns. For example, 'what is in the Reid project' would extract 'Reid' as a project filter, narrowing the search to only chunks from that project folder.

RAG Query Flow

For standard knowledge questions:

- I. Embed the question using the configured embedder
- II. If a project filter was extracted, search only within that project's chunks
- III. Otherwise, search across all chunks with optional `source_filter`
- IV. Retrieve top-5 most similar chunks with their metadata
- V. Format chunks into a numbered context block for Claude
- VI. Send context + question to Claude with a detailed system prompt
- VII. Append source attribution footer to Claude's response
- VIII. Return the complete answer

System Prompt for Claude

Claude is given specific instructions to ground its answers in the retrieved context and cite sources. The system prompt instructs Claude to be concise, highlight cross-document connections, and explain code files in plain language.

CLI Usage

```
# Basic query
python src/rag/rag_query.py "What does the Reid project do?"
```

```
# Filter to a specific project
python src/rag/rag_query.py "explain main.py" --project "Person-Reid-Model"
```

```
# Filter to a specific source
python src/rag/rag_query.py "what are my tasks" --source notion
```

```
# Trigger file send (requires chat ID)
python src/rag/rag_query.py "Send me the proposal" --chat-id 7165743149
```

```
# Weekly digest
python src/rag/rag_query.py --digest
```

```
# Knowledge base statistics
python src/rag/rag_query.py --stats
```

5.5 file_sender.py

This file implements the file delivery feature — one of the most distinctive capabilities of this system. When you need a document that is stored on Lightning AI, you simply ask for it in Telegram and it is sent directly to your chat.

Location

```
src/telegram/file_sender.py
```

Two-Stage Search Strategy

File finding uses a two-stage search to maximise recall:

Stage 1: Filename Keyword Match

The query is first stripped of action words (send me, share, get me, etc.) and then matched against filenames stored in ChromaDB metadata. If any filename contains words from the cleaned query, those files are returned as candidates ranked by how many words match.

Example: 'Send me the Project Proposal' → cleaned to 'project proposal' → matches 'Project Proposal.docx' (2/2 words match = score 1.0)

Stage 2: Semantic Vector Search

If no filename matches are found, the system falls back to semantic embedding search. The cleaned query is embedded and compared to all chunk embeddings in ChromaDB. Files whose chunks score above the minimum relevance threshold (0.10) are returned as candidates.

File Size Limit

The Telegram Bot API has a 50 MB limit for file uploads. This system enforces a stricter 25 MB limit. Files larger than 25 MB will not be sent; instead, the bot sends an error message with the file's server path.

Sending Process

Once a file is identified, it is uploaded to Telegram using the sendDocument API method. The file is read from disk as binary data and streamed directly to Telegram's servers in a multipart/form-data POST request. A caption is attached showing the filename, project name, and match score.

list_files_in_project

This function retrieves all unique files stored in ChromaDB metadata for a given project name and formats them as a readable list with filenames, types, and sizes. It optionally sends the list directly to a Telegram chat.

5.6 notion_ingest.py

This ingestion source connects to the Notion API and retrieves all pages accessible to your integration token. It is useful for ingesting meeting notes, task lists, project documentation, and research notes stored in Notion.

Location

```
src/ingest/notion_ingest.py
```

How It Works

- I. Connects to Notion using the NOTION_API_KEY from .env
- II. Fetches all pages accessible to the integration via the search endpoint
- III. For each page, retrieves all block content recursively
- IV. Converts blocks to plain text (handles paragraphs, headings, bullets, code, tables)
- V. Chunks, embeds, and stores with source='notion' metadata

Setup Requirements

To use Notion ingestion:

- I. Go to notion.so/profile/integrations and create a new integration
- II. Copy the integration token into .env as NOTION_API_KEY
- III. Share each Notion page or database with your integration (using the Share button in Notion)

```
python src/ingest/notion_ingest.py
```

5.7 obsidian_ingest.py

Obsidian is a popular markdown-based note-taking application. This ingestor walks an Obsidian vault directory and ingests all .md files, with special handling for Obsidian-specific syntax.

Location

```
src/ingest/obsidian_ingest.py
```

Obsidian-Specific Processing

- Wikilinks ([[Page Name]]) are converted to plain text: Page Name
- Tags (#tag) are stripped from content
- YAML frontmatter (---...---) is removed
- Embedded images (![image.png]) are stripped
- Internal links with aliases ([[Page|Alias]]) → Alias

Usage

```
# Point to your Obsidian vault directory
python src/ingest/obsidian_ingest.py --path /path/to/obsidian/vault
```

5.8 gmail_ingest.py

This ingestor uses the Gmail API with OAuth2 to fetch important email threads and make them searchable through your knowledge base.

Location

```
src/ingest/gmail_ingest.py
```

OAuth2 Setup (One-Time)

Gmail OAuth requires a one-time browser-based authentication that must be performed on a local machine (not headlessly on Lightning AI):

- I. Create a Google Cloud project at console.cloud.google.com
- II. Enable the Gmail API
- III. Create OAuth2 credentials and download credentials.json
- IV. Run the ingestor locally once: python src/ingest/gmail_ingest.py
- V. Complete the browser authentication flow
- VI. Upload the generated gmail_token.json to Lightning AI

Subsequent runs use the stored token and refresh automatically without browser interaction.

What Gets Ingested

The ingestor fetches threads labelled as IMPORTANT or STARRED. It retrieves the subject, sender, recipient, and body of each email in the thread, combining them into a single document for context preservation.

5.9 OpenClaw Skills (SKILL.md files)

OpenClaw reads SKILL.md files from the ~/.openclaw/skills/ directory to know what capabilities the agent has. Each skill file tells OpenClaw when to use the skill and what command to run.

rag-search/SKILL.md

This skill is triggered when the user asks knowledge questions. It runs the `rag_query.py` script with the user's question and returns the output.

```
Trigger examples: /rag_search What does X do?, What did I write about Y?  
Command: cd ~/project && python src/rag/rag_query.py "{{query}}"
```

file-send/SKILL.md

This skill is triggered when the user wants to receive a file. It runs `rag_query.py` with the `--chat-id` flag to enable file delivery.

```
Trigger examples: /file-send Project Proposal, send me the report  
Command: python src/rag/rag_query.py "{{query}}" --chat-id {{chat_id}}
```

universal-ingest/SKILL.md

This skill triggers re-ingestion from Telegram, useful when you have added new files and want to update the knowledge base without opening a terminal.

```
Trigger examples: /universal-ingest, ingest my files, update knowledge base  
Command: python src/ingest/universal_ingest.py
```

Installing Skills

Skills must be copied to the OpenClaw skills directory to take effect:

```
mkdir -p ~/.openclaw/skills/rag-search  
cp src/skills/rag-search/SKILL.md ~/.openclaw/skills/rag-search/  
  
mkdir -p ~/.openclaw/skills/file-send  
cp src/skills/file-send/SKILL.md ~/.openclaw/skills/file-send/  
  
mkdir -p ~/.openclaw/skills/universal-ingest  
cp src/skills/universal-ingest/SKILL.md ~/.openclaw/skills/universal-ingest/
```

5.10 AGENTS.md

The `AGENTS.md` file defines the persona, personality, and behavioural rules for the OpenClaw agent. It is the system prompt that Claude receives at the start of every conversation.

Location

```
config/agents/AGENTS.md (and copied to ~/.openclaw/agents/main/agent.md)
```

Key Sections

Identity

Defines the agent as KnowledgeOS — a personal knowledge assistant for a specific user. This personalisation helps the agent give appropriately scoped responses.

Primary Purpose

Instructs the agent to always run the RAG script before answering knowledge questions, rather than answering from Claude's own training data.

Core Skills

Lists the available skills so the agent knows what capabilities it has and when to use them.

Proactive Behaviours

Defines scheduled actions like the weekly Monday digest at 9am IST.

Chapter 6: Configuration Reference

6.1 Environment Variables (.env)

The .env file is the single source of truth for all configuration. Never commit this file to Git — it contains API keys and tokens that must be kept secret.

Required Variables

Variable	Description
ANTHROPIC_API_KEY	Claude API key from console.anthropic.com. Used for answer synthesis and image description. Format: sk-ant-api03-...
TELEGRAM_BOT_TOKEN	Bot token from @BotFather. Create a bot with /newbot command. Format: 1234567890:AAGJVPx...
NGROK_AUTHTOKEN	Auth token from dashboard.ngrok.com. Required to run ngrok tunnel. Format: 2abc123...
CHROMADB_PATH	Local directory path for ChromaDB data files. Default: ./chromadb
CHROMADB_COLLECTION	Name of the ChromaDB collection. Default: knowledge_os
OPENCLAW_PORT	Port for OpenClaw gateway HTTP server. Default: 18789
OPENCLAW_MODEL	Claude model for OpenClaw agent. Default: anthropic/claude-haiku-4-5

Embedding Variables (One Required)

Variable	Description
VOYAGE_API_KEY	Voyage AI key from dash.voyageai.com. Free 200M tokens. Highest priority if set.
OPENAI_API_KEY	OpenAI key from platform.openai.com. Paid. Second priority if set.
OLLAMA_BASE_URL	Ollama service URL. Set to http://localhost:11434 for local Ollama. Third priority.
OLLAMA_EMBED_MODEL	Ollama model name. Default: nomic-embed-text. Only used when OLLAMA_BASE_URL is set.

Optional Variables

Variable	Description
NGROK_URL	Fixed ngrok URL for paid ngrok accounts. If set, ngrok uses this domain instead of a random one.
NOTION_API_KEY	Notion integration token from notion.so/profile/integrations. Required for Notion ingestion.
GOOGLE_CLIENT_ID	Google OAuth client ID. Required for Gmail ingestion.
GOOGLE_CLIENT_SECRET	Google OAuth client secret. Required for Gmail ingestion.
OBSIDIAN_VAULT_PATH	Path to your Obsidian vault directory. Required for Obsidian ingestion.

6.2 openclaw.json

OpenClaw stores its configuration in `~/.openclaw/openclaw.json`. Key sections:

agents.defaults

Defines the default model and workspace for the main agent. Do not add keys that are not in OpenClaw's schema — it validates this file on startup.

gateway

Controls how the HTTP server binds. `bind: 'lan'` means port 18789 is accessible on the local network (0.0.0.0). `bind: 'loopback'` would restrict to localhost only.

tools.web.search

Controls web search behaviour. Set `enabled: false` to prevent OpenClaw from searching the internet instead of your knowledge base.

channels.telegram

Stores the bot token for the Telegram integration. This token is used to send and receive messages.

plugins

Enables and disables OpenClaw plugins. The `memory-lancedb` plugin is disabled on Lightning AI because it requires AWS Bedrock permissions that are not available.

6.3 ChromaDB Collection Settings

The ChromaDB collection is created once with specific settings that cannot be changed later without recreating it:

Setting	Value & Explanation
<code>hnsw:space</code>	<code>cosine</code> — uses cosine similarity for vector comparison (0=identical, 1=opposite)
<code>name</code>	<code>knowledge_os</code> — the collection name, set by <code>CHROMADB_COLLECTION</code> in <code>.env</code>

The HNSW (Hierarchical Navigable Small World) algorithm is ChromaDB's indexing method for fast approximate nearest-neighbour search. It builds a graph structure that allows logarithmic-time search over millions of vectors.

Chapter 7: Complete Setup Guide

7.1 Prerequisites

Before starting, create accounts and collect credentials from these services:

Service	What You Need	URL
Anthropic	API key (ANTHROPIC_API_KEY)	console.anthropic.com
Telegram	Bot token from @BotFather	t.me/BotFather
ngrok	Auth token + free account	dashboard.ngrok.com
Voyage AI (optional)	API key (VOYAGE_API_KEY)	dash.voyageai.com
Notion (optional)	Integration token	notion.so/profile/integrations
Lightning AI	Studio account	lightning.ai

7.2 Step-by-Step Installation

Step 1 — Clone Repository

```
git clone https://github.com/Abhishek242001/personal-knowledge-rag-os.git
cd personal-knowledge-rag-os
```

Step 2 — Configure Environment

```
cp .env.example .env
```

Open `.env` and fill in at minimum: `ANTHROPIC_API_KEY`, `TELEGRAM_BOT_TOKEN`, `NGROK_AUTHTOKEN`, `OLLAMA_BASE_URL` or `VOYAGE_API_KEY`, `CHROMADB_PATH`, `CHROMADB_COLLECTION`, `OPENCLAW_PORT`, `OPENCLAW_MODEL`.

Step 3 — Install Python Dependencies

```
pip install -r requirements.txt
```

Step 4 — Install Ollama

```
curl -fsSL https://ollama.ai/install.sh | sh
ollama serve &
sleep 5
ollama pull nomic-embed-text
curl http://localhost:11434/api/tags # Verify running
```

Step 5 — Install OpenClaw

```
npm install -g openclaw@latest
# If openclaw command not found:
find / -name 'openclaw.mjs' 2>/dev/null | head -3
alias openclaw='node /path/to/openclaw.mjs'
```

Step 6 — OpenClaw Onboarding (First Time Only)

```
openclaw onboard --mode local
```

Select: Model → anthropic/claude-haiku-4-5, Port → 18789, Bind → LAN, Channel → Telegram, Web search → DuckDuckGo.

Step 7 — Install Skills

```
mkdir -p ~/.openclaw/skills/rag-search ~/.openclaw/skills/file-send
~/.openclaw/skills/universal-ingest
cp src/skills/rag-search/SKILL.md ~/.openclaw/skills/rag-search/
cp src/skills/file-send/SKILL.md ~/.openclaw/skills/file-send/
cp src/skills/universal-ingest/SKILL.md ~/.openclaw/skills/universal-ingest/
```

Step 8 — Add Documents

Create project folders and add your files:

```
mkdir -p my_knowledge/MyProject/documents
# Copy your PDFs, Word docs, Excel files, code, images here
```

Step 9 — Run Ingestion

```
python src/ingest/universal_ingest.py
```

Expected output: confirmation of how many files were found and how many chunks were stored.

Step 10 — Start ngrok

```
mkdir -p logs
ngrok http 18789 > logs/ngrok.log 2>&1 &
sleep 3
curl -s http://localhost:4040/api/tunnels | python3 -c \
    "import sys,json; d=json.load(sys.stdin); print(d['tunnels'][0]['public_url'])"
```

Step 11 — Set Telegram Webhook

```
BOT_TOKEN=$(grep TELEGRAM_BOT_TOKEN .env | cut -d'=' -f2)
NGROK_URL=https://your-ngrok-url.ngrok-free.app
curl "https://api.telegram.org/bot${BOT_TOKEN}/setWebhook?url=${NGROK_URL}"
```

Step 12 — Start OpenClaw

```
export $(grep -v '^#' .env | xargs)
openclaw gateway
```

Expected: [gateway] ready and [telegram] starting provider (@your_bot_name)

Step 13 — Test

Message your bot on Telegram:

```
/rag_search What documents do I have?
/file-send Project Proposal
```

7.3 One-Command Startup Script

After initial setup, use this script to start everything in one command:

```
bash ~/start_bot.sh
```

The startup script performs these steps automatically:

- a) Installs Python dependencies (pip install -q ...)
- b) Installs and starts Ollama, pulls embedding model
- c) Installs OpenClaw via npm
- d) Loads environment variables from .env
- e) Updates the Anthropic API key in OpenClaw's auth-profiles.json
- f) Starts ngrok tunnel in background
- g) Sets Telegram webhook to new ngrok URL
- h) Starts OpenClaw gateway

Chapter 8: Usage Guide

8.1 Telegram Commands

Command	Description
<code>/rag_search What does X do?</code>	Search knowledge base and get an AI-synthesised answer
<code>/rag_search Explain main.py</code>	Ask about code files — Claude explains in plain language
<code>/rag_search Summarise the proposal</code>	Summarise any ingested document
<code>/file-send Project Proposal</code>	Receive the file directly in Telegram
<code>/file-send requirements.txt</code>	Works for any file type
<code>/universal-ingest</code>	Trigger re-ingestion of all files
<code>/new</code>	Start a fresh conversation session

8.2 Effective Query Techniques

Be Specific

Specific questions produce better results than vague ones. Compare:

- Vague: 'Tell me about the project'
- Specific: 'What detection model does the Person-Reid-Model use for person detection?'

Mention Project Names

If you know which project the information is in, mention it:

- 'In the Reid project, how are embeddings generated?'
- 'What does config.py in Person-Reid-Model configure?'

Ask for Summaries

- 'Summarise the Project Document from Reid'
- 'What are the key points in the project proposal?'

Cross-Document Questions

- 'Compare the approaches used in my two research papers'
- 'What libraries appear across all my Python projects?'

8.3 CLI Usage for Power Users

The RAG system can be used directly from the terminal without Telegram:

```
# Standard query
python src/rag/rag_query.py "What is the system architecture?"
```

```
# Filter to specific project
python src/rag/rag_query.py "explain the matching algorithm" --project "Person-Reid-Model"
```

```
# Filter to specific source
python src/rag/rag_query.py "my open tasks" --source notion
```

```
# Increase number of retrieved chunks (default 5)
python src/rag/rag_query.py "detailed question" --top-k 10
```

```
# Send file (replace with your chat ID from OpenClaw logs)
python src/rag/rag_query.py "Send me Project Proposal" --chat-id 7165743149
```

```
# Weekly digest
python src/rag/rag_query.py --digest
```

```
# Stats
python src/rag/rag_query.py --stats
```

Chapter 9: Troubleshooting Guide

9.1 Ollama Issues

Connection Refused Error

Symptom: ConnectionError: Cannot connect to Ollama at http://localhost:11434

```
# Start Ollama
ollama serve &
sleep 5
ollama pull nomic-embed-text
curl http://localhost:11434/api/tags # Should return JSON
```

404 Not Found on /api/embeddings

Symptom: HTTPError: 404 Not Found for url: http://localhost:11434/api/embeddings

Cause: Your Ollama version uses the new /api/embed endpoint. The codebase has been fixed to use the new endpoint, but if you see this error, check embedder.py line ~96 and ensure it uses /api/embed not /api/embeddings.

KeyError: 'embedding'

Symptom: KeyError: 'embedding' when calling the embed endpoint

Cause: Newer Ollama versions return 'embeddings' (plural, list of lists) instead of 'embedding' (singular, flat list). The fix is in embedder.py:

```
embeddings = data.get('embeddings') or data.get('embedding')
return embeddings[0] if isinstance(embeddings[0], list) else embeddings
```

9.2 OpenClaw Issues

Command Not Found

Symptom: zsh: command not found: openclaw

```
# Find the installation
find / -name 'openclaw.mjs' 2>/dev/null | head -3

# Run directly with node
node /path/to/openclaw.mjs gateway

# Or add permanent alias
echo 'alias openclaw="node /path/to/openclaw.mjs"' >> ~/.zshrc
source ~/.zshrc
```

Invalid API Key Error

Symptom: HTTP 401 authentication_error: invalid x-api-key in OpenClaw logs

```
export $(grep -v '^#' .env | xargs)
python3 -c "
import json, os
f = '/teamspace/studios/this_studio/.openclaw/agents/main/agent/auth-profiles.json'
with open(f) as fh: auth = json.load(fh)
auth['profiles']['anthropic:default']['key'] = os.getenv('ANTHROPIC_API_KEY')
```

```
with open(f, 'w') as fh: json.dump(auth, fh, indent=2)
print('Updated')
"
```

Config Validation Failed

Symptom: Invalid config: agents.defaults: Unrecognized key: 'systemPrompt'

Cause: OpenClaw validates its config schema strictly. Do not add keys that are not part of its schema. Run `openclaw doctor --fix` to repair.

9.3 Telegram Issues

Bot Not Responding

Check these in order:

- Is OpenClaw gateway running? (should see [gateway] ready in terminal)
- Is ngrok running? `curl http://localhost:4040/api/tunnels`
- Is webhook set? `curl https://api.telegram.org/botTOKEN/getWebhookInfo`
- Is API key valid? Check OpenClaw logs for 401 errors
- Did you message the correct bot? Check `@your_bot_name`

Bot Answers from Internet Instead of Knowledge Base

Cause: OpenClaw web search is enabled and finds web results faster than triggering your skill.

```
python3 -c "
import json
with open('/teamspace/studios/this_studio/.openclaw/openclaw.json') as f: c=json.load(f)
c['tools']['web']['search']['enabled'] = False
with open('/teamspace/studios/this_studio/.openclaw/openclaw.json','w') as f:
    json.dump(c,f,indent=2)
"
```

9.4 ChromaDB Issues

Dimension Mismatch

Symptom: Error when ingesting — dimension of existing collection does not match

Cause: You switched embedding providers. Ollama generates 768-dimensional vectors, Voyage AI generates 512-dimensional vectors. The collection cannot hold both.

```
# Delete collection and re-ingest
python3 -c "
import chromadb
c = chromadb.PersistentClient('./chromadb')
c.delete_collection('knowledge_os')
print('Deleted - re-run ingestion now')
"
python src/ingest/universal_ingest.py
```

Files Not Found by Ingestion

Symptom: ingestion finds 1 file when many were expected

Cause: Files may be inside a git submodule (empty on the server) or the folder structure does not match expectations.
Debug with:

```
python3 -c "  
from pathlib import Path  
for f in Path('my_knowledge').rglob('*'):  
    if f.is_file(): print(f)  
"
```

Chapter 10: Advanced Topics

10.1 Adding a New Ingestion Source

To add a new data source (e.g., Confluence, Jira, local databases), create a new file in `src/ingest/` following this pattern:

- Create `src/ingest/myservice_ingest.py`
- Implement a fetch function that returns a list of (text, metadata) tuples
- Use the shared `get_embedder()` and `get_collection()` functions
- Store with a unique `source=` value in metadata
- Create a corresponding skill in `src/skills/myservice/SKILL.md`

The key is to store a unique source identifier in the metadata so your new chunks can be filtered separately from others.

10.2 Switching Embedding Providers

Switching from Ollama to Voyage AI is recommended for Lightning AI users because Voyage AI is persistent across session restarts. However, you must re-ingest all documents after switching because the vector dimensions are different.

- Sign up at dash.voyageai.com
- Copy your API key
- Add `VOYAGE_API_KEY=your-key` to `.env`
- Delete the ChromaDB collection: `python3 -c "import chromadb; c=chromadb.PersistentClient('./chromadb'); c.delete_collection('knowledge_os')"`
- Re-run ingestion: `python src/ingest/universal_ingest.py`

All future ingestion and queries will now use Voyage AI automatically.

10.3 Increasing Retrieval Quality

If answers seem off-topic or miss relevant information, try these improvements:

Adjust chunk size

Smaller chunks (200 words) improve precision but may miss cross-sentence context. Larger chunks (600 words) improve context but may introduce noise. Experiment with `CHUNK_SIZE` in `universal_ingest.py`.

Increase top-K

The default retrieves 5 chunks. For complex questions spanning multiple documents, increase to 10: `--top-k 10`

Use project filtering

If your knowledge base is large, always mention the project name to limit the search scope.

Re-ingest with better chunking

If initial ingestion used poor chunking, delete the collection and re-ingest after adjusting parameters.

10.4 Scaling the System

As your knowledge base grows beyond tens of thousands of chunks, consider these optimisations:

- ChromaDB supports millions of vectors — no architectural changes needed up to ~10M chunks

- For better multi-project isolation, use separate ChromaDB collections per project
- For team use, consider moving ChromaDB to a shared persistent volume
- For higher concurrency, run multiple OpenClaw instances behind a load balancer

10.5 Security Considerations

This system handles personal and potentially confidential documents. Important security practices:

- Never commit `.env` to Git — it contains all your API keys
- The `.gitignore` file excludes `.env`, `chromadb/`, and `logs/` by default
- The ngrok tunnel exposes your system to the internet — ensure OpenClaw's gateway auth token is strong
- Gmail OAuth token (`gmail_token.json`) should never be committed
- OpenClaw's recommended permission settings: `chmod 700 ~/.openclaw` and `chmod 600 ~/.openclaw/openclaw.json`
- Consider using ngrok's IP allowlist feature to restrict webhook access to Telegram's IP ranges

Chapter 11: Maintenance & Operations

11.1 Adding New Files

Whenever you add new documents to my_knowledge/:

- a) Place files in the appropriate project subfolder
- b) Run ingestion: `python src/ingest/universal_ingest.py`
- c) Or trigger from Telegram: `/universal-ingest`

Ingestion is idempotent — re-ingesting existing files overwrites their chunks with the same content (based on deterministic MD5 IDs), so there is no need to delete first unless you changed the files.

11.2 Updating Existing Files

If you modify an existing document and want the knowledge base to reflect the changes:

- a) Delete the project's existing chunks: `python src/ingest/universal_ingest.py --clean --project ProjectName`
- b) Re-ingest: `python src/ingest/universal_ingest.py --project ProjectName`

The `--clean` flag deletes all chunks for the specified project before re-ingesting.

11.3 Monitoring

Check the health of the system with these commands:

```
# Knowledge base statistics
python src/rag/rag_query.py --stats
```

```
# OpenClaw live logs
tail -f /tmp/openclaw/openclaw-$(date +%Y-%m-%d).log
```

```
# ngrok status
curl -s http://localhost:4040/api/tunnels | python3 -m json.tool
```

```
# Telegram webhook status
curl https://api.telegram.org/botYOUR_TOKEN/getWebhookInfo
```

11.4 Backup

The ChromaDB data directory (`./chromadb/`) contains all your ingested knowledge. Back it up regularly:

```
tar -czf knowledge_backup_$(date +%Y%m%d).tar.gz chromadb/
```

The `.env` file should also be backed up securely (not to Git).

Chapter 12: Quick Reference Card

12.1 Most-Used Commands

Task	Command
Start everything	<code>bash ~/start_bot.sh</code>
Start Ollama only	<code>ollama serve & && sleep 5 && ollama pull nomic-embed-text</code>
Start OpenClaw only	<code>export \$(grep -v '^#' .env xargs) && openclaw gateway</code>
Ingest all files	<code>python src/ingest/universal_ingest.py</code>
Ingest one project	<code>python src/ingest/universal_ingest.py --project ProjectName</code>
Query from terminal	<code>python src/rag/rag_query.py "your question"</code>
Send file from terminal	<code>python src/rag/rag_query.py "Send me X" --chat-id 7165743149</code>
Check DB stats	<code>python src/rag/rag_query.py --stats</code>
Clear a project	<code>python src/ingest/universal_ingest.py --clean --project ProjectName</code>
Clear entire DB	<code>python3 -c "import chromadb; c=chromadb.PersistentClient('./chromadb'); c.delete_collection('knowledge_os')"</code>
View OpenClaw logs	<code>tail -f /tmp/openclaw/openclaw-\$(date +%Y-%m-%d).log</code>
Fix API key	See Chapter 9.2
Update webhook	<code>curl https://api.telegram.org/botTOKEN/setWebhook?url=NGROK_URL</code>

12.2 Telegram Command Reference

Telegram Command	What It Does
<code>/rag_search <question></code>	Search knowledge base and get AI answer
<code>/file-send <filename></code>	Receive a file directly in Telegram
<code>/universal-ingest</code>	Re-ingest all files in my knowledge/
<code>/new</code>	Start a fresh session (clears conversation context)

12.3 Environment Variable Quick Reference

Variable	Required / Example
<code>ANTHROPIC_API_KEY</code>	Required — sk-ant-api03-...
<code>TELEGRAM_BOT_TOKEN</code>	Required — 1234567890:AAG...
<code>NGROK_AUTHTOKEN</code>	Required — 2abc123def...
<code>VOYAGE_API_KEY</code>	Optional (recommended) — pa...
<code>OLLAMA_BASE_URL</code>	Required if no VOYAGE/OPENAI — http://localhost:11434
<code>CHROMADB_PATH</code>	Required — ./chromadb
<code>CHROMADB_COLLECTION</code>	Required — knowledge_os
<code>OPENCLAW_PORT</code>	Required — 18789
<code>OPENCLAW_MODEL</code>	Required — anthropic/claude-haiku-4-5
<code>NOTION_API_KEY</code>	Optional — ntn_...

Chapter 13: Glossary

Term	Definition
RAG	Retrieval-Augmented Generation — AI technique that retrieves relevant documents before generating an answer
Vector Embedding	A list of floating-point numbers that represents the semantic meaning of a piece of text
ChromaDB	A local vector database that stores embeddings and supports similarity search
Cosine Similarity	A measure of similarity between two vectors based on the angle between them (0=identical, 1=opposite)
Chunk	A small segment of a document (typically 400 words) used as the unit of storage and retrieval
Chunking	The process of splitting documents into smaller overlapping segments
HNSW	Hierarchical Navigable Small World — ChromaDB's graph-based index for fast vector search
OpenClaw	A self-hosted AI agent gateway that connects Telegram to custom Python skills
Webhook	An HTTP callback — Telegram uses it to push messages to your server in real time
ngrok	A tool that creates a public HTTPS tunnel to a private local server
Ollama	A local LLM and embedding model runner — runs AI models entirely on your own hardware
omic-embed-text	An open-source embedding model (768 dimensions) available via Ollama
Voyage AI	A cloud embedding service optimised for use with Claude (512 dimensions, free 200M tokens)
Claude	Anthropic's large language model — used for answer synthesis and image description
Skill	An OpenClaw markdown file that defines a capability: what triggers it and what command to run
Ingestion Pipeline	The offline process of reading, chunking, embedding, and storing documents
Query Pipeline	The online process of answering user questions via retrieval and generation
Metadata	Structured data stored alongside each chunk: filename, project, path, type, etc.
Source Filter	A metadata filter that restricts ChromaDB search to chunks from a specific ingestion source
Project Filter	A metadata filter that restricts ChromaDB search to a specific project folder
Semantic Search	Search based on meaning and context rather than exact keyword matching
LLM	Large Language Model — an AI model trained on text to understand and generate language

Appendix A: Complete .env.example

Copy this file to .env and fill in your credentials:

```
# — Required —
ANTHROPIC_API_KEY=sk-ant-api03-your-key-here
TELEGRAM_BOT_TOKEN=your-bot-token-from-botfather
NGROK_AUTH_TOKEN=your-ngrok-auth-token

# — Embeddings (choose one) —
VOYAGE_API_KEY=pa-your-voyage-key           # Recommended
# OPENAI_API_KEY=sk-your-openai-key         # Alternative
# OLLAMA_BASE_URL=http://localhost:11434    # Local alternative
# OLLAMA_EMBED_MODEL=nomic-embed-text

# — ChromaDB —
CHROMADB_PATH=./chromadb
CHROMADB_COLLECTION=knowledge_os

# — OpenClaw —
OPENCLAW_PORT=18789
OPENCLAW_MODEL=anthropic/claude-haiku-4-5

# — Optional knowledge sources —
NOTION_API_KEY=ntn-your-notion-key
NGROK_URL=https://your-fixed-ngrok-url.ngrok-free.app
```

Appendix B: requirements.txt

```
anthropic>=0.25.0      # Claude API
chromadb>=0.4.0        # Vector database
python-dotenv>=1.0.0   # .env loading
requests>=2.31.0       # HTTP calls
pymupdf>=1.23.0        # PDF extraction
python-docx>=1.1.0     # Word documents
python-pptx>=0.6.23    # PowerPoint
pandas>=2.0.0          # Excel and CSV
openpyxl>=3.1.0        # Excel backend
xlrd>=2.0.1           # Legacy .xls
notion-client>=2.2.0   # Notion API
google-auth>=2.28.0    # Gmail OAuth
google-auth-oauthlib>=1.2.0
google-api-python-client>=2.120.0
# voyageai>=0.2.3      # Voyage AI (optional)
# openai>=1.12.0       # OpenAI (optional)
```



Uniconverge Technologies Pvt. Ltd.

uniconvergetech.in

Developed by Abhishek Kumar Shukla — Senior AI Developer

linkedin.com/in/researcher-abhishek-kumar-shukla